

Software Systems Design (2024) - Architecture

Assignment 2

(40 Points)

You are responsible to design a software architecture for the following brownfield system. You should use the Attribute Driven Design (**ADD**) method and document the final architecture in multiple views. In this assignment, **FOUR** students need to form a group to collectively complete the assignment. The score of each student on this assignment will be determined with the group performance by considering individual contribution to as well as reflection on the work.

DEADLINE: 12:00am on (Monday) June 3, 2024

The BankStat Case Study provided as the appendix reading for this assignment presents an existing architecture for the system used by the Atlantic Bank.

Assignment Deliverables

A single PDF file should be submitted per group. Your report should be in TWENTY pages (A4 size) or shorter and include the following deliverables:

- A report on how you have followed the Attribute Driven Design method (20 points). You have to perform at least **TWO** iterations (10 points per iteration) when designing the architecture. This part includes two components:
 - A list of important non-functional requirements (and constraints) in the format of scenarios that you have identified or assumed.
 - A list of all the architecturally significant requirements (ASRs), design concerns, and architectural patterns/tactics you chose every time when you performed Step 3 (choose an ASR) and 4.2/4.3 (choose architectural patterns/tactics) of the ADD method. This should include all intermediary candidates that might not have been included in the final architecture. A *brief* explanation of the choice should be provided.
- Final software architecture documentation (15 points):
 - Views (models) and cross-views following the template of Views & Beyond (Chapter 18 in 3rd Edition or Chapter 22 in 4th Edition) [1]. There should be at least **THREE** views (with at least one module view) plus one cross-view. The “rationale” part cannot be omitted. (For more information, see [2])

- You are encouraged to use suitable UML diagrams. (For more information, see [3]) However, you may instead choose to use other alternative modeling language notations that you feel more comfortable with.
- Individual remark should be completed by each student (about half page each) in a group (5 points), and contains two components:
 - A description of your personal experiences using the ADD method.
 - A summary of your own contributions to the group work and report.

Notes:

- 1. You can compile and submit your report in either Chinese or English. You are not necessarily 100% correct in language when using English. However, you should make sure you convey everything in a clear and understandable manner.*
- 2. When you require more information on requirements or affecting factors to make a design decision, you could make a reasonable assumption, document it and then make your design decision.*
- 3. There is no minimum length requirement for this assignment, but it should not exceed TWENTY pages. Be concise and to the point.*
- 4. You will be assessed based on your ability to apply the design method and document the design rather than the absolute correctness of your design.*

[1] Software Architecture in Practice, 3rd or 4th Edition (reprint from the English edition published by Tsinghua University Press or China Machine Press)

[2] Documenting Software Architectures: Views and Beyond, 2nd Edition (some e-book available on Internet)

[3] UML Distilled: A Brief Guide to the Standard Object Modeling Language, 2nd Edition (some e-book available on Internet)

Appendix

Bank Statement System

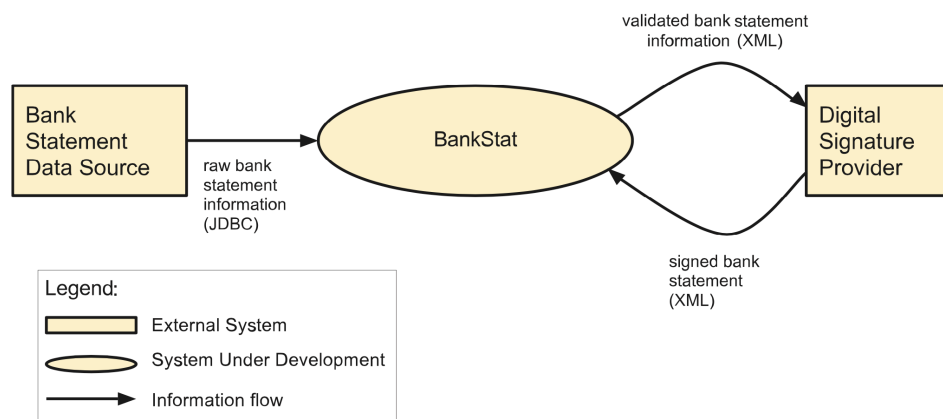
(a case study)

Note: this is a simplified and generalized description of a real system

Business Case

Recently, the International Banking Organization (IBO) issued a regulation that requires banking institutions to digitally sign bank statements. To comply with the regulation, Atlantic Bank decided to commission the development of a software system, which is called BankStat, whose main purpose is the generation of digitally signed bank statements.

The context diagram below illustrates how the BankStat system works. At its core, the system executes a batch process, which retrieves raw bank statement information from a data source (an external database) and then performs a series of validations on this data to generate the bank statements and prepare them for digital signature by an external provider. The statements are sent to the provider, which returns the signed bank statements. These statements are then stored by BankStat for further processing, including sending the statements to customers.



This batch process is triggered automatically once a month and, during its execution, approximately 2 million bank statements are processed. The following quality attributes scenarios are primary for this system:

- **Reliability:** Under normal operating conditions, the batch process is executed in its entirety 100% of the time.
- **Performance:** Under normal operating conditions, when the batch process starts, 2 million bank statements are read, processed, and sent to the signing provider in at most one hour.

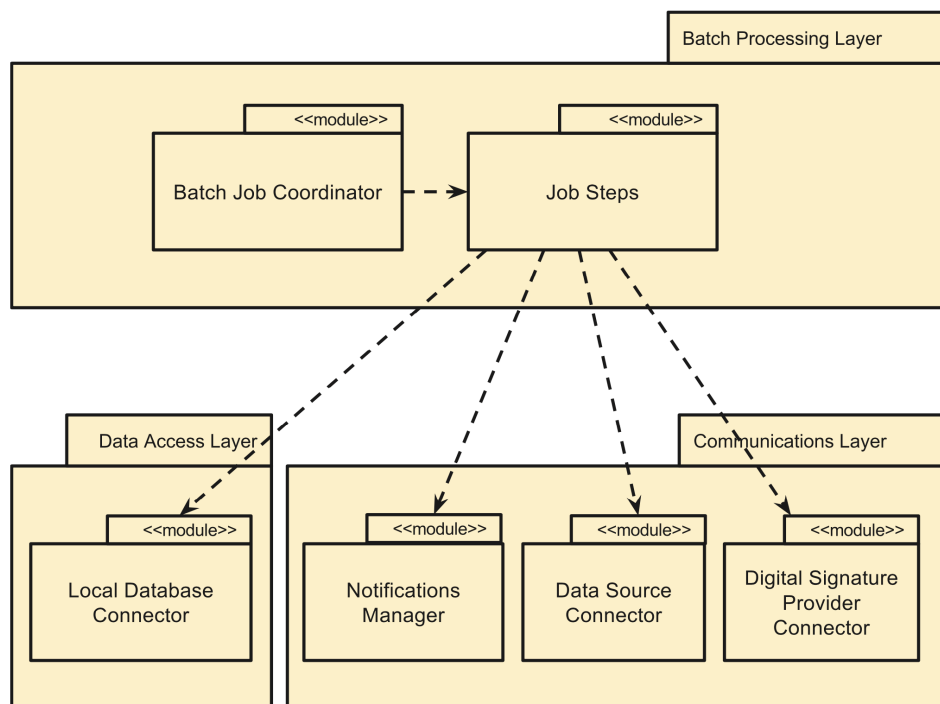
- **Availability:** During normal processing, a failure may occur when reading information from the data source or when sending information for digital signature. A notification is then sent to the administrator, who manually restarts the process. When it is restarted, only the information that had not already been processed is treated.

Due to time constraints imposed by the IBO, only the core batch process for the system was developed and put into production. This initial release, however, did not provide a friendly interface with the system, which is necessary to monitor the state of the bank statement processing, to request the reprocessing of incorrect statements, and to generate reports. In the first release, the process could only be started or stopped manually from a console. For a second release of the system, the Atlantic Bank requests an extension of the BankStat system to better address these shortcomings.

Existing System Architecture

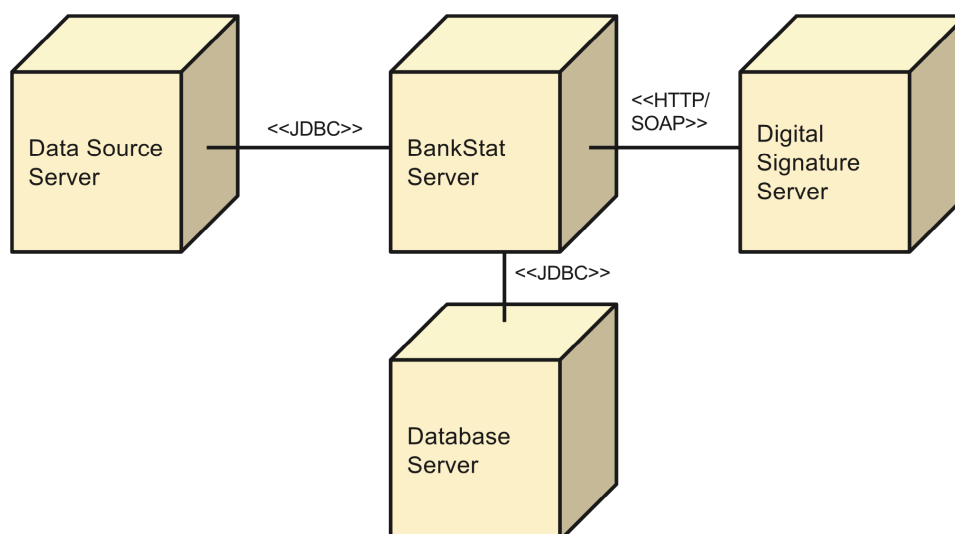
This section presents a simplified version of the existing system's (the initial release) view examples, which provide relevant information for the changes in the architecture.

Module View



Element	Responsibility
Batch Processing Layer	This layer contains modules that perform the batch process. These components are developed using the Spring Batch framework.
Data Access Layer	This layer contains modules that store and retrieve data from a local database, which is used by the modules in the Batch Processing Layer.
Communications Layer	This layer contains modules that support communication with the external digital signature provider and the bank statement data source.
Batch Job Coordinator	This module is responsible for coordinating the execution of the batch process, including launching the process and invoking the different steps associated with it.
Job Steps	This module contains the “steps” that are part of the batch job. These steps perform activities such as validating the information retrieved from the data source and generating the bank statements. Such steps generally read, process, and write data. Data is read from and written to the local database.
Local Database Connector	This module is responsible for accessing a local database used by the job steps to exchange information while performing the batch process. We refer to this database as “local” to differentiate it from the external data source; this database is used only locally (i.e., internally) by the application, even if it is deployed in a different node (see the next section).
Notifications Manager	This module manages logs and sends notifications in case of issues such as a communication failure with the external system.
Data Source Connector	This module is responsible for connecting with the external database that provides the raw bank statement information.
Digital Signature Provider Connector	This module is responsible for accessing the external system that performs the digital signing of the bank statements.

Allocation View

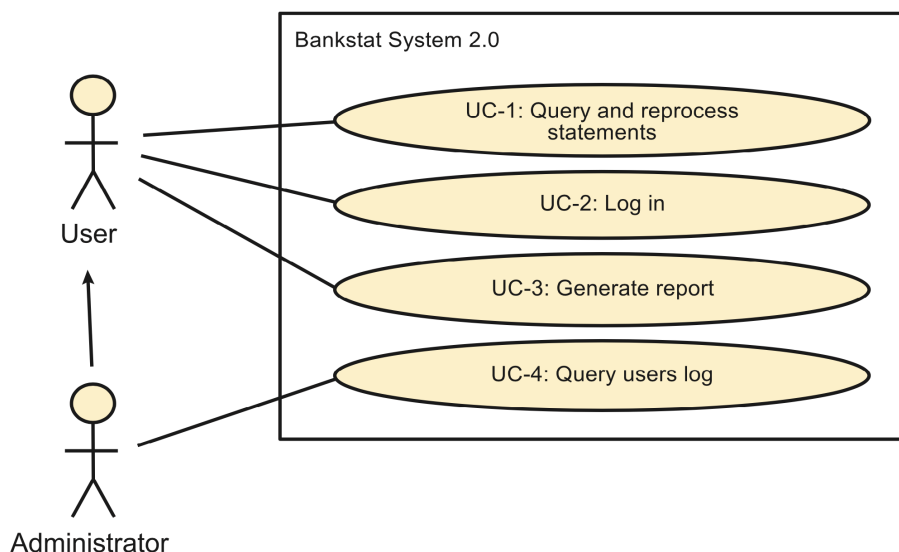


Element	Responsibility
Data Source Server	This server hosts a database that contains the raw data used to produce the bank statements.
BankStat Server	This server hosts the main batch process that is responsible for retrieving information from the Data Source Server, validating the information, and sending the information to the Digital Signature Server for signing.
Database Server	This server hosts a database that is used locally by the batch process in the BankStat Server to hold the state and information used in the execution of the batch process.
Digital Signature Server	This server, which is provided by an external entity, is responsible receiving, digitally signing, and returning the bank statements. The server exposes web services that receive and produce XML information.

Drivers for Second Release

Use Case Model

Use Case	Description
UC-1: Query and reprocess statements	The user manually requests the reprocessing of a number of statements. The user specifies criteria to query and select the statements that must be reprocessed. The user can, for example, select a period of interest or status of the statements that he is interested in (e.g., processed, signed, non-signed).
UC-2: Log in	The user logs in to the system.
UC-3: Generate report	The user generates reports regarding the process.
UC-4: Query users log	The administrator queries user logs to display the activities of a particular user or groups of users. Information can be filtered using criteria such as dates or types of operations.



Quality Attribute Scenarios

ID	Quality Attribute	Scenario	Associated Use Case
QA-1	Security	A user performs any operation on the system, at any moment, and 100% of the operations performed by the user are recorded by the system in the operations log.	UC-4

Constraints

ID	Constraint
CON-1	The user's accounts and permissions are handled by an existing user directory server that is used by various applications in the bank.
CON-2	Communication with the data source must be realized using JDBC.
CON-3	Communication with the digital signature provider system is performed using web services. These web services receive and return the information in an XML format that adheres to specifications established by the government.
CON-4	The system must be accessed from a web browser, although the access is available only from the bank's intranet.

Architectural Concerns

ID	Concern
CRN-1	The system shall be programmed using Java and Java-related technologies to leverage the expertise of the development team.
CRN-2	The introduction of new functionality must, as far as possible, avoid modifications to the existing batch processing core.